REPORT 007: REPORT 007: EPOCH 7, KONDUKTIVA-PEAK LIVE-CODING INTEGRATION, 8-PLAYER POLYPHONIC & DUAL-CC SWEEP SEQUENCER BRIDGE, AND CROSS-LANGUAGE RUST/NODE PARITY

2026-11-06 03:31:00 | konduktiva-peak Live-Coding Integration, 8-Player Polyphonic & Dual-CC Sweep Sequencer Bridge, and Cross-Language Rust/Node Parity

Operator: Antigravity (Gemini 3.8 Flash) · Follows: report-006.md

---

Prompt

```
[FORMAL INTENT]: Epoch 7 of 8: konduktiva-peak Live-Coding Integration, 8-Player
↪  Polyphonic & Dual-CC Sweep Sequencer Bridge, and Cross-Language Rust/Node Parity
↪  (Target: src/konduktiva_bridge.rs; Related: js/konduktiva-peak,
↪  tests/cross_synth_parity.rs). Integrate js/konduktiva-peak (createPeakKit(K),
↪  sweepCC, dualSweepCC, rhythmPresets, noteVoice, ccLane, createPeakPlayers,
↪  runStressTest, safeSweepRange, midi-cc-map.json) with js/shared-synth-sdk and
↪  Rust src/konduktiva_bridge.rs + GUI Konduktiva live-coding tab displaying the 8
↪  canonical Peak players (bass, chords, lead, arp, ccCutoff, ccReso, ccMod, ccFx),
↪  density graphs (peakHold, peakBuild), and live CC lane telemetry. Add
↪  cross-language integration tests in tests/cross_synth_parity.rs and
↪  js/konduktiva-peak/test/run.js verifying byte-identical CC/NRPN/SysEx,
↪  safeSweepRange clamping, OSC, and preset roundtrips between Rust and Node.js.
↪  Ensure all workspace tests pass (CARGO_BUILD_JOBS=2 cargo test --workspace and
↪  npm test). Write reports/report-007.md and compile it with
↪  /home/renick/claude/html-report-generator/build-report.sh --pdf.
```

TL;DR

Epoch 7 implements the complete Konduktiva live-coding companion subsystem and cross-language integration between Rust and Node.js. In src/konduktiva_bridge.rs, we implemented the 8-player polyphonic voice allocation and continuous CC sweep sequencer bridge (4 note players: bass, chords, lead, arp; 4 CC lane players: ccCutoff, ccReso, ccMod, ccFx) with density graphs (peakHold and peakBuild), triangular CC sweeps, and real-time telemetry tracking. In src/app.rs and src/gui.rs, we added the dedicated Konduktiva live-coding navigation tab with real-time transport controls, visual 16-beat density curve canvas, player strip cards, and real-time oscilloscope sparkline telemetry for all 4 CC lanes with safe clamping limits enforced. In tests/cross_synth_parity.rs and js/konduktiva-peak/test/run.js, we added extensive cross-language parity test suites verifying byte-identical 74 CC mappings, 32 bipolar dual-CC pairs, 14-bit NRPNs, safeSweepRange clamping, OSC address patterns and packet encoding, Novation SysEx frame structure and 7-bit checksums, and preset JSON roundtrips. All 188 automated tests (162 Rust tests and 26 Node.js tests) pass with zero compiler warnings.

1. Overview and Everyday Purpose

In live algorithmic music performance (often called live-coding), an artist generates and modifies music in real time by writing and running software commands while an audience listens. Rather than pre-recording static audio tracks, the performer commands automated musical players that trigger melodies, chords, rhythm patterns, and continuous synthesizer dial sweeps.

The Novation Peak is an eight-voice polyphonic hybrid synthesizer. Because it is monotimbral (it plays one overall patch across its eight hardware voices on MIDI Channel 1), managing a live performance requires careful coordination. If all voices fire at once, notes can cut off awkwardly, or extreme filter resonance can create sudden speaker-damaging volume spikes.

Epoch 7 solves these challenges by integrating the `konduktiva-peak` live-coding driver across both the native Rust desktop application and the Node.js ecosystem. To understand how this works, consider these everyday parallels:

1. The Eight-Musician Chamber Ensemble: Imagine an ensemble with four instrumentalists playing musical notes (a double bass player, a rhythm chord keyboardist, a solo melodic lead player, and a fast arpeggiator) alongside four sound technicians continuously turning tone knobs (one sweeping the filter brightness, one adjusting resonance, one moving the modulation wheel, and one adjusting reverb and echo effects). Our sequencer bridge manages all eight performers simultaneously, ensuring note chords and sound sweeps harmonize cleanly without exceeding the eight-voice hardware limit.
2. The Musical Energy Accelerator (Density Graphs): In a concert, a band may hold a steady groove before building up excitement ahead of a crescendo. Our density graph system allows the live performer to switch between `peakHold` (where each voice stays on its calm, native tempo) and `peakBuild` (where the density of musical events smoothly doubles across a 16-beat cycle before relaxing back down).
3. The Oscilloscope Dashboard (Live CC Lane Telemetry): When a technician monitors a power station, live meters show voltage waves to catch power spikes before fuses blow. Our live telemetry dashboard draws real-time vector oscilloscope sparklines for each control lane, graphing dial movements and visually highlighting the green safety clamp limits that prevent runaway resonance or distortion.
4. Universal Translator (Cross-Language Parity): If two architects collaborate on a building using two different CAD software packages, every measurement, blueprint symbol, and steel specification must match down to the millimeter. Our cross-language parity suite verifies that both Rust and Node.js agree on every single MIDI controller number, parameter address, safety limit, and preset file format.

We define all specialized industry concepts on first use:

- Polyphony: The ability of an electronic instrument to play multiple distinct musical notes at the same time. The Novation Peak supports up to eight simultaneous voices.
- Monotimbral: An instrument that plays one shared sound patch across all its voices at any given moment, rather than playing different instrument sounds on separate channels.
- Continuous Controller (CC): A standard MIDI message used to adjust a synthesizer dial or slider (such as volume, filter cutoff, or effect depth) using values from 0 to 127.
- Inter-Onset Interval (IOI): The duration of time or number of beats between the start of one musical note and the start of the next note.
- Density: The frequency of musical events occurring per beat. A density of 0.5 means one note every two beats (half notes), whereas a density of 4.0 means four notes per beat (sixteenth notes).
- Non-Registered Parameter Number (NRPN): An extended 14-bit MIDI addressing method that controls parameters beyond the standard 128 Continuous Controllers, providing 16,384 distinct parameter values.
- System Exclusive (SysEx): Manufacturer-specific MIDI data packets used to transfer complete sound presets and hardware settings between a computer and a synthesizer.

- Open Sound Control (OSC): A high-speed network protocol that transmits musical control messages over local UDP computer networks with human-readable text addresses like `/peak/filter/cutoff`.
- Telemetry: Automated remote measurement and collection of operational data, presented as real-time metrics and waveform charts.

The following table maps these everyday musical goals directly to their code components:

Everyday Musical Goal	Technical Implementation	Code Path and Symbol
Coordinate 8 musical performers	Canonical 8-Player Sequencer Bridge	`KonduktivaBridge`, `src/konduktiva_bridge.rs`
Maintain 8-voice polyphonic budget	4 Note Voices + 4 CC Lanes Layout	`NOTE_PLAYER_NAMES`, `CC_PLAYER_NAMES`, `src/konduktiva_bridge.rs`
Smooth triangular knob sweeps	Mathematical Triangle Wave Generator	`tri`, `SweepCc`, `DualSweepCc`, `src/konduktiva_bridge.rs`
Steady vs rising musical energy	Density Graphs (`peakHold` & `peakBuild`)	`DensityGraphMode`, `src/konduktiva_bridge.rs`
Dedicated live-coding GUI tab	6th Navigation Tab & Controls	`MainTab::KonduktivaLive`, `render_konduktiva_panel`, `src/gui.rs`
Real-time dial telemetry waveforms	Live Oscilloscope Sparkline Buffers	`CcLaneTelemetry`, `src/konduktiva_bridge.rs`
Hardware acoustic protection	Automated Safe Clamping Limits	`safe_sweep_range`, `synth_midi_osc::routing`
Emergency sound silence	Panic Button & Packet Emission	`KonduktivaBridge::panic`, `src/konduktiva_bridge.rs`
Verify byte-for-byte Rust/JS parity	Comprehensive Integration Test Suite	`tests/cross_synth_parity.rs`, `js/konduktiva-peak/test/run.js`

---

2. The Eight Canonical Peak Players and Sequencer Bridge (‘src/konduktiva_bridge.rs‘)

In `src/konduktiva_bridge.rs`, we built a complete native sequencer bridge that mirrors the architecture of `js/konduktiva-peak`. Because the Peak has an eight-voice limit, running multiple independent musical patterns could easily cause voice-stealing artifacts. Our bridge uses four polyphonic note voices and four continuous controller sweep lanes, all communicating over wire MIDI Channel 0 (hardware MIDI Channel 1):

2.1 The Four Polyphonic Note Voices

1. `bass` (Octave 2, native rate 0.5 onsets per beat): Emits steady root and fifth basslines using velocities of 98 and 90 with a generous note duration of 1.7 beats.
2. `chords` (Octave 4, native rate 0.5 onsets per beat): Emits three-note and four-note minor triads and minor seventh chords (`[0, 3, 7]`, `[3, 7, 10]`, `[5, 8, 12]`, and `[7, 10, 14]`) with gentle velocities of 66 and 60.
3. `lead` (Octave 5, native rate 2.0 onsets per beat): Plays an expressive melodic contour in eighth notes drawn from the C natural-minor scale pool (`[0, 2, 3, 5, 7, 8, 10, 12]`) with velocities cycling through 104, 88, 96, and 84.
4. `arp` (Octave 5, native rate 4.0 onsets per beat): Performs rapid sixteenth-note arpeggios that

dart across chord intervals with dynamic velocity accents (80, 64, 72, and 60) and crisp 0.2-beat durations.

Together, these four voices consume at most six simultaneous voices during peak overlaps, remaining safely within the eight-voice hardware limit.

2.2 The Four Continuous CC Sweep Lanes

To give the performance organic movement, four lanes continuously modulate the Peak hardware dials using deterministic triangular sweeps:

1. `ccCutoff` (CC 29, native rate 4.0 onsets per beat): Sweeps the analog filter cutoff frequency from 28 to 118 across an 8-beat cycle in 32 discrete steps.
2. `ccReso` (CC 79, native rate 2.0 onsets per beat): Sweeps filter resonance from 6 to 62 across a 6-beat cycle in 24 steps. The upper ceiling is clamped strictly at 62 to prevent harsh self-oscillation.
3. `ccMod` (CC 1, native rate 2.0 onsets per beat): Modulates the hardware modulation wheel from 0 to 100 across a 4-beat cycle in 16 steps.
4. `ccFx` (Dual sweep on CC 112 and CC 108, native rate 1.0 onsets per beat): Emits two synchronized parameter updates per onset across a 12-beat cycle. Controller A sweeps Reverb Level (CC 112) between 20 and 90, while Controller B sweeps Delay Level (CC 108) between 12 and 70 with a one-third cycle phase offset (`steps / 3`).

2.3 Mathematical Triangle Sweep Formula

In both Rust and Node.js, continuous parameter sweeps are calculated using a pure mathematical triangle wave function that scales linearly from 0.0 to 1.0 and returns back to 0.0:

$$\text{tri}(i, \text{steps}) = 1.0 - \left| \frac{2 \cdot i}{\text{steps}} - 1.0 \right|$$

This symmetric function guarantees smooth, periodic parameter modulation without abrupt discontinuities or audio pops.

---

3. Density Graphs: ‘peakHold‘ and ‘peakBuild‘

Musical rhythm is governed by density curves that dictate how many notes occur each beat. The bridge implements two canonical density graph modes:

3.1 ‘peakHold‘ (Steady Native Rungs)

Under `peakHold`, the rate multiplier remains constant at 1.0x for all players. Each voice sits on its natural rhythmic rung:

- Bass and Chords: 0.5 onsets per beat (half-note pace).
- CC FX Lane: 1.0 onsets per beat (quarter-note pace).
- Lead, CC Resonance, and CC Mod: 2.0 onsets per beat (eighth-note pace).
- Arp and CC Cutoff: 4.0 onsets per beat (sixteenth-note pace).

This creates an organized, polyrhythmic foundation that can run indefinitely without rhythmic crowding.

3.2 ‘peakBuild‘ (16-Beat Dynamic Energy Ramp)

When the performer engages `peakBuild`, the engine modulates the rate of all eight players simultaneously according to a 16-beat cycle:

$$\text{Multiplier}(\text{beat}) = \begin{cases} 1.0 + \frac{\text{beat}}{8.0}, & 0 \le \text{beat} \le 8 \\ 1.0 + \frac{16.0 - \text{beat}}{8.0}, & 8 < \text{beat} \le 16 \end{cases}$$

At beat 0, all voices start at their standard 1.0x rate. As the bar progresses, density steadily doubles, reaching 2.0x at beat 8 (where sixteenth notes accelerate into thirty-second notes and half notes become quarter notes), before relaxing smoothly back to 1.0x at beat 16. This provides an instant way to build tension and release during a live set.

---

4. Desktop GUI Konduktiva Live-Coding Tab (‘src/gui.rs‘ and ‘src/app.rs‘)

We added a sixth navigation tab, `MainTab::KonduktivaLive` (”6. Konduktiva Live”), to the native `eframe` desktop application, accessible via the number key 6. The panel provides comprehensive visual monitoring and manual performance controls:

4.1 Transport and Density Control Header

- Play and Stop Toggle: A high-visibility button that toggles between green (”PLAY”) and red (”STOP”).
- Tempo Slider: An interactive slider allowing tempo adjustment from 60.0 BPM to 240.0 BPM.
- Beat and Bar Readout: Displays real-time playhead progress in the format `Beat 04.25 (Bar 2.1)`.
- Reset Playhead: Instantly resets beat position back to 0.0 and clears pending note buffers.
- Density Mode Selectors: Clickable buttons to switch between `peakHold` and `peakBuild`, displaying the active multiplier readout (e.g., `Active Density Multiplier: 1.50x`).
- Emergency Panic Button: Immediately sends All-Notes-Off (CC 123), All-Sound-Off (CC 120), and resets the modulation wheel to silence all hardware sound.

4.2 Density Curve Canvas Visualizer

The GUI features a custom immediate-mode vector painter that visualizes the 16-beat density cycle:

- A dark slate background with vertical grid guidelines indicating each 4-beat bar line.
- A 64-point vector curve drawn in the active theme accent color (Oxford Blue or Amber) showing the current shape of the density curve.
- A bright red vertical playhead line that tracks real-time progress across the 16-beat cycle.

4.3 Eight Canonical Player Strip Cards

Two columns present the eight canonical players:

- Left Column (4 Note Voices): Shows active checkboxes, voice names, octave assignments, current rate readouts, and running event counters.

- Right Column (4 CC Sweep Lanes): Shows active checkboxes, parameter labels, target CC numbers, sweep range bounds, and running event counters.

4.4 Live CC Lane Telemetry and Oscilloscope Sparklines

At the bottom of the tab, four telemetry cards monitor parameter health:

1. `ccCutoff` (CC 29 Filter Cutoff): Displays live value, min/max limits, safe operating range `[0..127]`, and a 64-sample ring-buffer sparkline.
2. `ccReso` (CC 79 Filter Resonance): Displays live value, min/max limits, a prominent green "Safe: [0..62]" badge, and a sparkline confirming the value never exceeds 62.
3. `ccMod` (CC 1 Modulation Wheel): Displays live value, min/max limits, safe range `[0..100]`, and an oscilloscope sparkline.
4. `ccFx` (Dual Reverb CC 112 and Delay CC 108): Displays both live values simultaneously, safe ranges `[20..90]` and `[12..70]`, and two overlaid sparklines drawn in cyan and amber.

---

5. Cross-Language Rust/Node.js Parity Verification

To guarantee that the Rust application and Node.js packages remain in perfect synchronization, we created an extensive integration test suite in `tests/cross_synth_parity.rs` and expanded `js/konduktiva-peak/test/run.js`:

5.1 Byte-Identical 74 CC Mappings and 32 Dual-CC Pairs

We verified all 74 entries from `midi-cc-map.json` against `NovationPeakProfile::device_profile()` in Rust and `NOVATION_PEAK_PROFILE` in JavaScript:

- Filter Cutoff: CC 29.
- Filter Resonance: CC 79.
- Modulation Wheel: CC 1.
- Reverb Level: CC 112.
- Delay Level: CC 108.
- Pre-Filter Overdrive: CC 80.
- Post-Filter Drive: CC 36.
- Distortion Level: CC 104.
- Delay Feedback: CC 110.
- Chorus Level: CC 105.
- Animate 1 Hold: CC 114.
- Animate 2 Hold: CC 115.
- Arpeggiator Clock Gate: CC 116.
- 32 Bipolar Dual-CC Pairs: MSB controllers 0 through 31 paired with LSB controllers 32 through 63.

5.2 14-Bit NRPN Addressing

We verified that extended parameter controls map identically:

- Filter Type: MSB 0, LSB 48.
- Filter Slope: MSB 0, LSB 49.
- Oscillator 1 Waveform: MSB 0, LSB 64.
- Oscillator 2 Waveform: MSB 0, LSB 80.
- Oscillator 3 Waveform: MSB 0, LSB 96.
- The Rust MIDI router generates the exact 4-message packet sequence: CC 99 (MSB), CC 98 (LSB), CC 6 (Data MSB), and CC 38 (Data LSB).

5.3 Safe Sweep Range Clamping Parity

Both environments enforce identical safe ceilings:

- Resonance: Clamped to `[0..62]` (prevents harsh runaway).
- Distortion Level: Clamped to `[0..48]` (prevents hard clipping).
- Pre-Drive and Post-Drive: Clamped to `[0..50]` (protects speakers).
- Delay Feedback: Clamped to `[0..70]` (prevents infinite build-up).
- FX Levels: Clamped to `[0..90]`.
- Cutoff and General CCs: Full range `[0..127]`.

5.4 OSC Network Protocol Parity

We verified bidirectional OSC addressing across both environments:

- Inbound address paths `/peak/osc/{1..3}/*`, `/peak/filter/*`, `/peak/env/*`, `/peak/lfo/*`, `/peak/fx/*`, `/peak/animate/{1,2}`, and `/peak/panic`.
- Safety clamping during OSC dispatch: an inbound OSC message requesting full resonance (float 1.0) automatically clamps to integer 62 before transmission.

5.5 SysEx Codec, 7-Bit Checksums, and Universal 0x7F Bypass

We verified the Novation SysEx frame structure:

- Frame Header: `0xF0` followed by Focusrite/Novation manufacturer ID `[0x00, 0x20, 0x29]`, Device ID `0x00`, and Sound Dump command `0x00`.
- 16-Byte Patch Name: Padded with ASCII spaces.
- 7-Bit Modulo Checksum: Computed using the formula:

$$\text{Checksum} = (128 - (\text{Sum} \pmod{128})) \ \& \ 0x7F$$

- Universal Bypass: The validator accepts `0x7F` as a universal bypass checksum value.
- Frame Terminator: Ends with `0xF7`.

5.6 Preset JSON Roundtrip Schema Validation

A complete `SynthPatch` was serialized to JSON in Rust, validated against `SYNTH_PATCH_SCHEMA` in Node.js, and deserialized back into Rust without data loss.

---

6. Verification Results and Test Metrics

Every test across the Rust workspace and Node.js packages passed with zero compiler warnings and zero errors.

6.1 Rust Workspace Test Results (‘CARGO_BUILD_JOBS=2 cargo test --workspace‘)

Test Suite / Binary	File Path	Tests Passed	Failed	Status
`novation_peak_manager` (Core Library)	`src/lib.rs`	57	0	PASS
`novation_peak_manager` (CLI Binary)	`src/main.rs`	0	0	PASS
`cross_synth_parity` (Integration Suite)	`tests/cross_synth_parity.rs`	9	0	PASS
`synth_core` (Shared Ecosystem Crate)	`crates/synth-core/src/lib.rs`	34	0	PASS
`synth_midi_osc` (MIDI/OSC Routing Crate)	`crates/synth-midi-osc/src/lib.rs`	33	0	PASS
`synth_ui_theme` (Themes & Widgets Crate)	`crates/synth-ui-theme/src/lib.rs`	29	0	PASS
Total Rust Workspace Tests		**162**	**0**	**PASS**

6.2 Node.js Test Suite Results (‘npm test‘)

Test Suite / Script	Verification Target	Checks Passed	Failed	Status
`js/konduktiva-peak/test/run.js`	Driver, Sweeps, Parity & Capture Run	18	0	PASS
`test/baseline.test.js`	Package Loading & Identity	1	0	PASS
`test/novation-peak-profile.test.js`	74 CCs, 32 Dual CCs, NRPNs, Schemas	7	0	PASS
Total Node.js Tests & Checks		**26**	**0**	**PASS**

6.3 Combined Workspace Total

Across the entire workspace, **188 automated tests** executed successfully with zero failures, confirming complete functional correctness and cross-language parity.

Test coverage (188 total)

QA verdict

PASS

Report 007 | Generated 2026-11-06 03:31:00 | reports/report-007.html